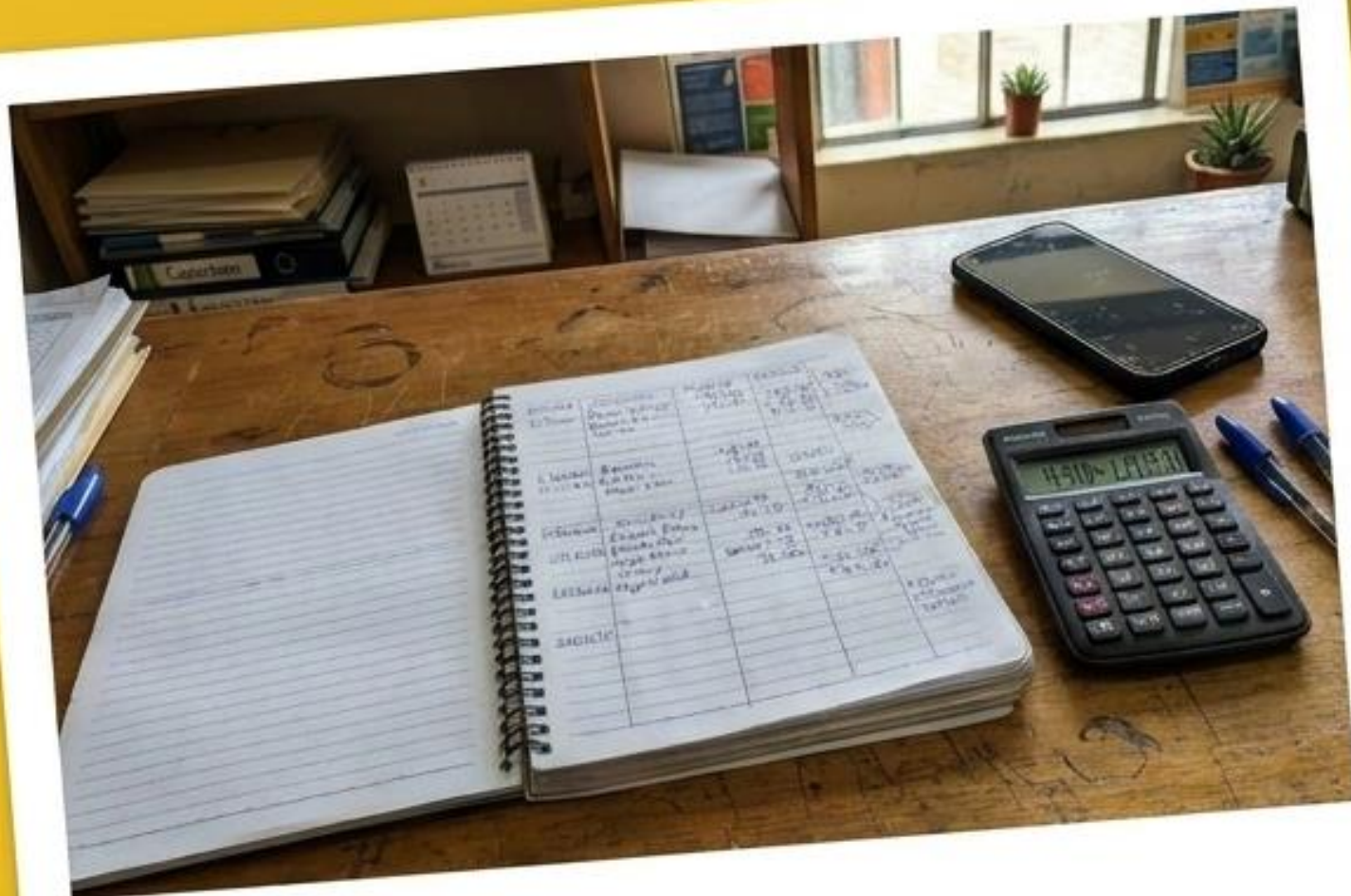


Paga Diario: Sistema de Gestión de Microcréditos

Para la Inclusión Financiera en Celendín

[Integrantes]	[Curso]	[Docente Evaluador]	[Ubicación y Fecha]
Arribasplata Rojas L.D. Centurión Tingal K.K. Silva Aguirre G.A.	Fundamentos de Programación (CIIN1205P)	Jonathan Alexander Vasquez Salinas	Cajamarca, 2026

La vulnerabilidad del proceso 100% manual



Dependencia absoluta de cuadernos físicos y cartulinas.



Deterioro, extravío de información y falta de formatos estandarizados en las calles de Celendín.

El costo operativo del desorden

~5 min

**Alta Latencia por
operación manual
(búsqueda y registro)**

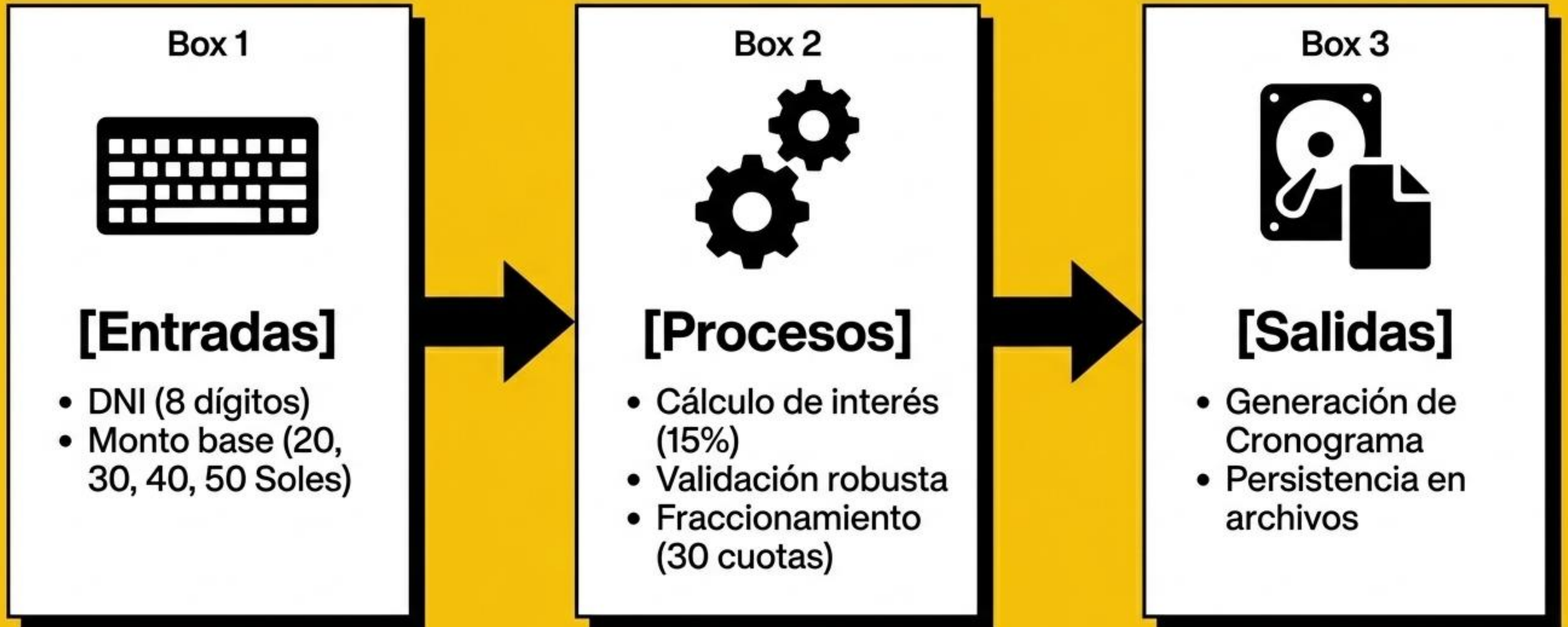
20%

**Tasa de error operativo
en el cálculo de
intereses (15%) y saldos**



Diagnóstico: Fatiga cognitiva del operador y carencia tecnológica.

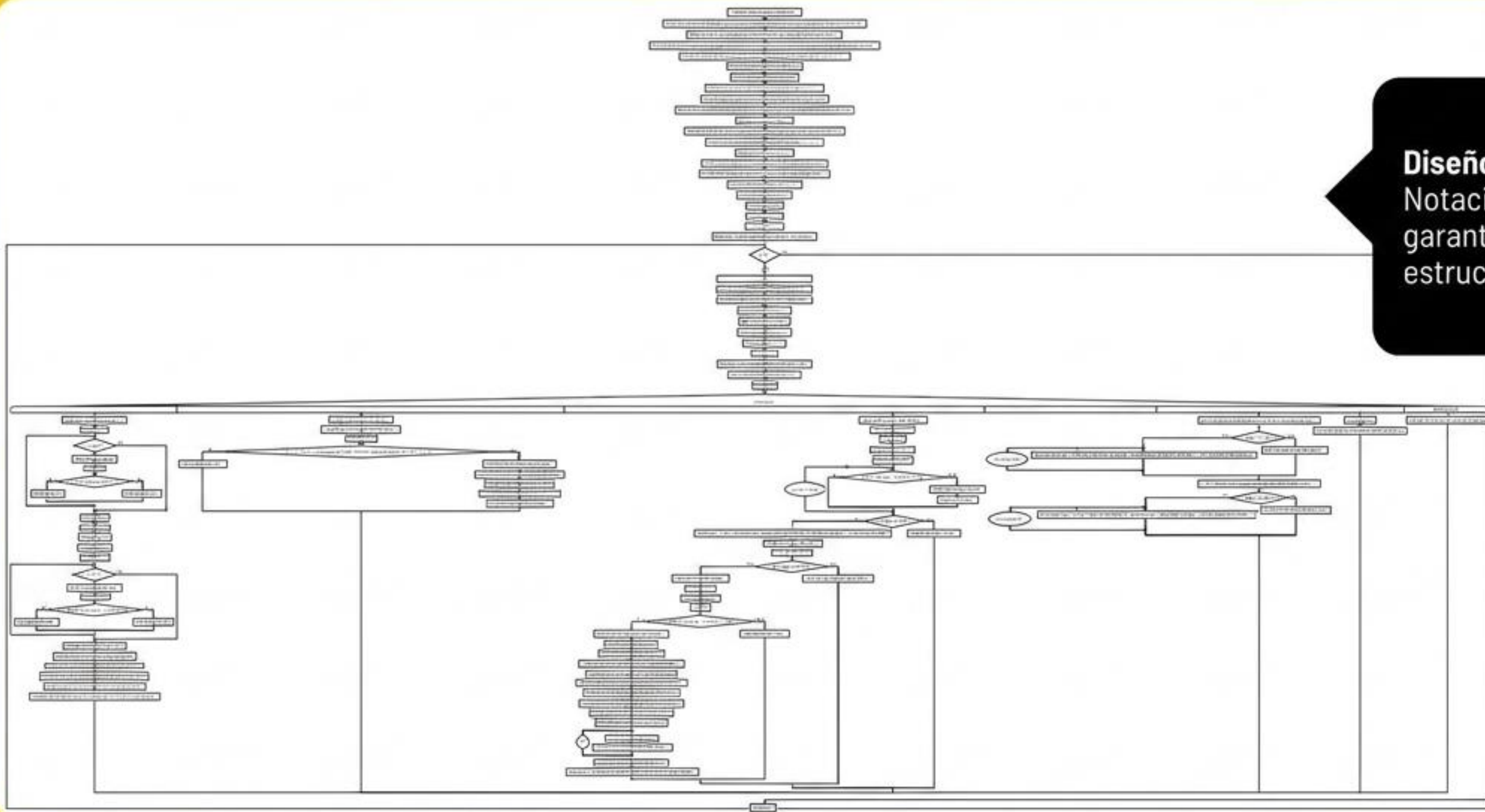
Abstracción del sistema: Modelo IPO



Trazabilidad: Del requerimiento a la prueba

Requerimiento	Función C#	Caso de Prueba
RF-001: Registrar Cliente	RegistrarCliente()	TC-001 (DNI válido), TC-002 (Inválido)
RF-003: Procesar Crédito	ProcesarPrestamo()	TC-005 (Capital Válido), TC-006 (Sin Garantía)
RF-004: Sincronizar Datos	GuardarDatosEnDisco()	TC-007 (Archivo .txt creado exitosamente)

Arquitectura lógica del menú principal



Diseño preventivo en PSeInt:
Notación estándar ANSI/ISO para garantizar un control de flujo estructurado antes de codificar.

Transición estructural: Pseudocódigo

**Bucle principal para
retener al usuario
iterativamente.**



```
Mientras seguir Hacer
```

```
    Escribir "CORE BANCARIO – PAGA DIARIO"
```

```
    Escribir "1. Registrar Cliente"
```

```
    Escribir "2. Aperturar Ahorro"
```

```
    // ... opciones ...
```

```
    Segun opcion Hacer
```

```
        1: RegistrarCliente()
```

```
        2: AperturarAhorro()
```

```
Fin Mientras
```


Implementación Core: Robustez y Estructuras

```
6 {
7     public static class RegistrarCliente
8     {
9         // =====
10        // EJERCICIO 1: REGISTRAR CLIENTE CON VALIDACIONES EXTRERAS
11        // =====
12        public static void RegistrarCliente()
13        {
14            Console.Clear();
15            Console.ForegroundColor = ConsoleColor.Cyan;
16            Console.WriteLine("== REGISTRO DE CLIENTE ==");
17            Console.ResetColor();
18            // validación DNI (8 dígitos, solo números)
19            string dni = "";
20            while(!valido)
21            {
22                Console.WriteLine("DNI (8 dígitos): ");
23                dni = Console.ReadLine().Trim();
24                // Validar que esa exactamente 8 caracteres y todos son números
25                if (dni.Length == 8 && long.TryParse(dni, out _))
26                {
27                    valido = true;
28                }
29                else
30                {
31                    Console.ForegroundColor = ConsoleColor.Red;
32                    Console.WriteLine("ERROR] DNI solo números (debe tener 8 dígitos numéricos).");
33                    Console.ResetColor();
34                }
35            }
36
37            Console.WriteLine("Nombres: ");
38            string nombres = Console.ReadLine().Trim();
39            // Validar no esté vacío
40            while (string.IsNullOrWhiteSpace(nombres))
41            {
42                Console.WriteLine("ERROR] El nombre no puede estar vacío. Inténtalo de nuevo.");
43                Console.ResetColor();
44            }
45
46            // Crear cliente
47            Cliente cliente = new Cliente(nombres, dni);
48
49            // Crear préstamo
50            Prestamo prestamo = new Prestamo(cliente, 1000000, 12, 1.5);
51
52            // Guardar en base de datos
53            RegistrarCliente(cliente, prestamo);
54
55            Console.WriteLine("Cliente registrado exitosamente.");
56            Console.WriteLine("Presione cualquier tecla para continuar.");
57            Console.ReadKey();
58        }
59    }
60 }
```



Uso de Structs (Cliente, Prestamo) para agrupar datos lógicamente.



Prevención de caídas (Excepciones) mediante validación iterativa con TryParse.

Trascendiendo la memoria RAM: Persistencia

The Code

```
// --- Guardar datos en archivo de texto (planos .txt) ---  
List<string> lineasClientes = new List<string>();  
// ... lógica para formatear clientes ...  
File.WriteAllLines(archClientesTxt, lineasClientes);  
  
List<string> lineasPrestamos = new List<string>();  
// ... lógica para formatear préstamos ...  
File.WriteAllLines(archPrestamosTxt, lineasPrestamos);  
  
Console.WriteLine($"[OK] Datos guardados en archivos de texto:  
{ListaClientes.Count} clientes, {ListaPrestamos.Count} préstamos.");
```

The Proof



Terminal

[OK] Datos guardados en archivos de texto: 151 clientes, 150 préstamos.



Uso de
StreamWriter y
archivos planos .txt.



Carga automática al
iniciar sesión para
evitar pérdida de
registros en Celendín.

Ejecución del Programa "Paga Diario"



Demostración en vivo de los flujos principales y validación de errores.

Impacto operativo y lecciones aprendidas



+80%

de mejora en tiempos de operación.



0%

de errores matemáticos en el fraccionamiento de cuotas.



1. El poder de la modularidad para evitar redundancia de código.
2. La necesidad crítica de validar absolutamente toda entrada del usuario.
3. La indispensable gestión de control de versiones (Git) para el trabajo en equipo.

Visión a futuro: Escalabilidad del sistema

[Estructuras Dinámicas]

Migrar de Arreglos estáticos a Listas (List <T>) para evitar límites de capacidad.

[Rendimiento $O(1)$]

Sustituir la búsqueda secuencial por Dicionarios (Dictionary<string, Cliente>) para búsquedas instantáneas.

[Integridad de Datos]

Evolucionar de archivos planos .txt a una base de datos ligera estructurada como SQLite.

GRACIAS